Adaptive schedules and exact kernels for subexponential SAT contraction

A second review of the combined research frontier

Research memorandum prepared for Vladimir Reshetnikov

27 October 2026

Abstract

This second review includes seven initial arrivals and five additional packets at a fixed closing snapshot. The latter already contain adaptive alphabets, streamed separators, binary contraction trees, and quadratic Cauchy tables. We credit those results and compare against their compatible combinations, not just their original isolated constants.

Under the unchanged normalized contraction interface, composing streamed conditioning with an explicit schedule gives polynomial-space time coefficient $55.3316221205\ldots$ on $s/\sqrt{\log_2 s}$, with an exact upper certificate below 55.35. The corresponding late-packet compositions have coefficients $55.4046659066\ldots$ and $55.4090551361\ldots$. A separate CRT/tree schedule has time coefficient $35.0706136705\ldots$ and space coefficient $33.9995016442\ldots$; for degree at most s^{51}, the polynomial-space coefficient is $25.1637654152\ldots$. These are resource-qualified constant improvements, not a better logarithmic SAT order or global optimization theorem.

The surviving practical improvements are support-sparse exact packing scores and fused transition ratios inside the already quadratic Cauchy recurrence. The latter reduces table multiplications from $(5N^2+N)/2$ to N^2+N, with linear setup and one unit inversion in both methods. Matched component experiments retain small-case regressions and charge full preparation. No measurement is a SAT solve time. Deformation, integrality, recovery, and recursive tensor interfaces remain inherited assumptions; this report is not a new formal verification or an independent proof of the claimed ETH refutation.

Contents

1 Conclusions and comparison discipline . . . 2
2 Sources and the combined baseline . . . 3
3 Closing intake and corrected comparison scope . . . 5
4 Normalized contraction interface and inherited lemmas . . . 7
5 A streamed polynomial space schedule . . . 9
6 A refined cached contraction tree schedule . . . 12
7 A degree sensitive streamed schedule . . . 14
8 Sparse exact conditional expectation updates . . . 15
9 Cauchy recurrence reproduction and new ratio fusion . . . 17
10 Finite evidence and measured performance . . . 20
11 Reproducibility and artifact map . . . 23
12 Limitations and next research decisions . . . 24
A Archive identities and source anchors . . . 25
B Why the geometric allocation has its stated form . . . 26

1 Conclusions and comparison discipline

The useful improvement is a portfolio, not one unqualified number. Five closing arrivals invalidate several novelty claims that would have been defensible at the initial snapshot; Section 3 documents the correction before presenting the final comparisons. The new polynomial-space schedule improves a normalized leading exponent coefficient. The two exact arithmetic kernels reduce work in explicitly specified component workloads. The higher-memory construction must be retained as a separate choice when memory is available. Its fair comparator includes retuning to a two-stage alphabet, not merely copying the original schedule. Low-degree specializations must likewise be retained for their narrower input class.

Throughout the report, a coefficient C means a bound of the form

$$T(s, \mathcal{L}) \le (s + \mathcal{L})^{O(1)} 2^{(C+o(1))s/\sqrt{\log_2 s}}, \tag{1}$$

under the same normalized wire interface. It is not a coefficient on SAT variables n without accounting for the reduction's wire count, degree, and description length. A smaller C is a smaller explicit upper-bound coefficient, not evidence that one implementation is faster on every finite input.

The supplied manuscripts still support a density-sensitive general envelope of the form

$$\mathrm{poly}_k(n, m)\, 2^{O_k\left(\frac{n \min\{\log(2+m/n), \log\log n\}}{\sqrt{\log n}}\right)}. \tag{2}$$

Our new schedule changes a normalized constant inside this pipeline, not the power of $\log n$. The independent Claude crossing-decomposition manuscript has a different implementation and unspecified comparable leading constants. No numerical dominance over that algorithm follows from a wire coefficient here.

Normalized route	Time C	Space C	Degree
Late fixed separator schedule	55.709478	Polynomial	$s^{\cdot 9}$
Late adaptive schedule plus separator	55.404666	Polynomial	$s^{\cdot 9}$
New explicit streamed composition	55.331622	Polynomial	$s^{\cdot 9}$
Late cached tree schedule	35.354588	34.274803	$s^{\cdot 9}$
New cached tree retuning	35.070614	33.999502	$s^{\cdot 9}$
Retained lower-memory option	37.324611	33.623041	$s^{\cdot 9}$
New degree-sensitive streamed route	25.163765	Polynomial	$s^{\cdot 51}$
Late very-low-degree cached route	15.391933	14.250161	$s^{1/12}$

Table 1: Final resource-qualified portfolio at the closing cutoff. Coefficients use $s/\sqrt{\log_2 s}$, not SAT variables. The rows are explicit supplied or composed certificates, not global optima.

1.1 What is new and what is inherited

- **Explicit schedule refinements.** Compose the late streamed evaluator with a strictly feasible rational schedule, and retune the late CRT/tree schedule. All paid child calls and memory maxima are included. Adaptive alphabets, conditioning, and the matrix backend themselves are prior contributions.
- **Support-sparse packing.** Avoid scoring inactive rows and unoccupied cells, preserving every exact potential and lexicographic tie. The stronger baseline is the late dense quadratic update, not only historical cubic scoring.

- **Fused Cauchy transitions.** Cache normalized transition ratios and absorb the common target prefactor once. The quadratic recurrence is inherited; the new comparison isolates this additional constant-factor arithmetic saving.
- **Unchanged assumptions.** Packing, taper, mode integrality, frozen precision, deformation, tensor factorization, recovery, and reduction identities remain explicitly imported. Finite tests are component evidence, not a full proof.

1.2 Resource frontiers cannot be conflated

The new polynomial-space coefficient 55.331622 is not a faster unrestricted-time bound than the cached alternatives. The new 35.070614/33.999502 cached certificate uses subexponential space, while the retained 37.324611/33.623041 option uses somewhat less memory. Lower-degree inputs can have stronger bounds, including the late 15.391933/14.250161 cached specialization for degree at most $s^{1/12}$. No one row dominates every resource and premise combination.

The scalar gains are in explicit upper-bound coefficients. We do not claim a universal practical speed ratio, an optimal schedule, or numerical dominance over the separate Claude algorithm's unspecified comparable constants.

2 Sources and the combined baseline

The initial reviewed repository snapshot is

`ab2fbc9e03d5c3d0ad6dec8686960bfe498b0b06`.

The target is the `Computability/SubexponentialSAT` research tree and its incoming documents. The earlier review covered the two primary manuscripts and seven earlier packets. This iteration adds the seven archives in Table 2. At that initial snapshot, the prior mathematical research tree was unchanged apart from those arrivals. Five further closing arrivals and later file relocations are treated separately below. The old report is useful provenance, but its headline constant is no longer an adequate benchmark.

Archive TeX, selected source files, and supporting notes were inspected statically. No program shipped inside an incoming archive was executed or imported. The executable implementations and finite checkers in this bundle were authored independently from the displayed formulas. This distinction avoids treating a packet's own test suite as independent verification.

2.1 The seven additional arrivals

The replacement factor is $\sqrt{3}/2$ for slope-three rows. The recursive-refinement row already uses slope 64/21, so its factor is $\sqrt{(64/21)/4}$ instead. Applying $\sqrt{3}/2$ to that row would manufacture an invalid gain. These combinations preserve mode-family multiplicity and child topology; the mode improvement cannot then be counted a second time through annulus refinements.

Among the initial seven arrivals, the *asymmetric packet* supplies the strongest generic packing analysis. It keeps the two side degrees separate: retained parent tensors have degree at most Δ, whereas newly made packet tensors have degree at most twice the previous packet scale. This permits a substantially larger label budget below the root. Its implementation already caches powers and future products, so a fair practical comparison must preserve those optimizations.

Arrival or stated family	Supplied C	Joint-compatible C
Asymmetric fixed alphabet	76.908995	66.605143
Asymmetric boundary family	76.432774	66.192724
Balanced packing	94.748202	82.054350
High moment packing	94.748202	82.054350
Packet refinement	95.663940	82.847403
Recursive refinement	133.207372	116.272927
Round two SAT refinement	84.066271	72.803527
Joint Cauchy with old schedule	319.685757	Already slope four

Table 2: Generic polynomial-space normalized certificates, not SAT-variable constants. A boundary value is an infimum of a family, not an attained finite-slack schedule. Joint-compatible means replacement of the mode basis while retaining that row's packing and schedule.

The *balanced packing* report attains the earlier 94.7482 boundary through a polynomial two-stage exact-rational construction. The *high-moment* report attains the same coefficient by a different construction. Its growing moment $t = \lceil \log_2 s \rceil$ incurs quasipolynomial setup $2^{O((\log s)^2)}$, explicitly paid as lower order; this must not be described as polynomial setup. Its fixed-moment variant has a coefficient penalty $\sqrt{(t+1)/t}$.

The *packet refinement* overlaps the previous matching-based packing and additive-gap improvements, but is not the new frontier. The *recursive refinement* improves variance packing and uses residue-sensitive modes; its generic coefficient remains weaker. Its degree regrouping is a separate promise. The *round two* packet improves the schedule and exact crossing fraction without changing the logarithmic order. The *joint Cauchy* packet's principal value is its full-block basis and memory tradeoff, not the older packing constants it retains for comparison.

2.2 The earlier seven packets and primary manuscripts

The earlier material already contains factorial Cauchy modes, exact Hermite tapering, revised recovery precision, streamed packet extraction, and density-sensitive reductions. In particular, the leading child precision $6MD/u$ is not new here. The density packets already improve some growing-density regimes to the numerator in (2). Two stronger suggested asymptotic routes in the earlier packets depend on unproved compact-factorization or degree-preserving-recursion interfaces. We do not silently promote these proposed interfaces to theorems.

The primary OpenAI manuscript's general and bounded-density rates are stronger than a weaker displayed formalization scope bound with an extra $\log \log n$ factor. Improving that weaker statement alone would not improve the combined manuscript frontier. The separate Claude construction has its own crossing-decomposition assumptions and claimed verification scope; neither its comparable constants nor its complete formal development have been independently established here.

2.3 Initial composed polynomial space comparators

For a constant alphabet $r = 16$, the asymmetric boundary family and joint modes yield

$$C_{16} = \frac{\pi}{\ln 2}\left(\sqrt{150} + \frac{\sqrt{50}}{3}\right) = 66.1927236936\ldots. \tag{3}$$

But fixing the same alphabet forever is unnecessarily restrictive. An admissible two-stage boundary family uses root alphabet 17 and tail alphabet 13. Set

$$A_0 = \frac{1}{20\log_2 17}, \qquad A = \frac{1}{10\log_2 13}, \qquad c = \frac{2}{13}, \qquad \kappa(r) = \frac{\pi\sqrt{2(1-1/r)}}{\ln 2}.$$

Optimally distributing the remaining leading budget $A - A_0$ geometrically gives

$$C_{17,13} = \frac{\kappa(17)}{\sqrt{A_0}} + \frac{(2/17)\kappa(13)}{\sqrt{A - A_0}\,(1 - c^{2/3})^{3/2}} = 66.120475\ldots. \tag{4}$$

This is a stronger comparator than (3), but it is not the final closing-snapshot frontier. Our intermediate tuple-enumeration coefficient 63.758994 is about 3.57% lower; streamed conditioning below supplies the stronger final comparison. We make no claim that either this two-stage family or our adaptive family is globally optimal over all admissible schedules, packings, or algorithms.

3 Closing intake and corrected comparison scope

A final fixed-cutoff review found five additional archives at commit

`fa6e904d109b738154db7fe41e7f2777e6d24aee`.

They materially change the comparison. This section supersedes any novelty or best-frontier interpretation that would follow from the initial seven-arrival review alone. The report does not repeatedly move the finish line: this is the fixed closing snapshot. The earlier packets were also relocated into the research collection without a mathematical change; source identities, not directory names, determine which versions are being compared.

Closing arrival and resource regime	Time C	Space C
`sat_adaptive_contraction`, streamed	55.709478	Polynomial
Same packet, adaptive cached	36.750313	34.024178
`subsat_contraction_tree`, generic	35.354588	34.274803
`subexponential_sat_frontier`	63.852203	Polynomial
Polynomial space		
Same packet, cached	37.930109	33.623041
`sat_frontier_improvements`	63.857409	Polynomial
Polynomial space		
Same packet, cached	38.484496	34.274803

Table 3: Newly supplied generic normalized certificates at the fixed closing cutoff. The fifth archive contributes instance-sensitive and extraction kernels, not a stronger generic coefficient. The rows have different memory requirements; no scalar ranking substitutes for that distinction.

3.1 Improvements that must now be treated as prior work

The late `sat_frontier_improvements` packet already contains the same consecutive-mode Cauchy recurrence, triangular skipping, exact valuation/unit state, linear-size denominator difference set, and batch inversion. It even absorbs the prefactor denominator into the inverse batch, requiring one unit inversion instead of our initial prototype's two. Our initial cubic-to-quadratic experiment is therefore

an independent reproduction and validation, not a new improvement over the closing corpus. Its raw measurements remain valid against its specified product reference, but cannot establish superiority over the late quadratic implementation. The late contraction-tree packet independently gives the streaming prefix formulation as well.

Depth-dependent alphabets are also already present in two closing packets. The initial coefficients 63.758994 and 37.591703 in our derivations are further explicit retunings, not invention of adaptive alphabet scheduling. Moreover, the new 55.709478 polynomial-space and 35.354588 cached certificates are stronger time bounds than those initial retunings. They must be combined with useful new schedules before claiming an improved final frontier.

The late `subexponential_sat_frontier` packet already reduces packing score work from cubic to quadratic in the label count. Its dense candidate update maintains future products, cell powers, and row sums, but still scans all labels and rows. The surviving new packing refinement is narrower: subtract the label-independent part, accumulate only positive occupied-cell support, and use positivity to select the first absent label exactly. Consequently our *dense-delta* implementation, not cubic rescoring, is the relevant stronger practical comparator. The observed 1.53–2.73 ratios on the three four-per-row fixtures are measured against that stronger independently implemented formula. Small-case regressions remain part of the evidence. No incoming code was executed.

3.2 Additional compatible components retained in the portfolio

The kernel-improvements packet supplies minimum-valuation elimination on the actual real-port crossing matrix over a finite unramified chain ring. Its Smith weights dominate the weights of every integral rank-one decomposition of the same matrix, so the feasible mode-tuple inventory cannot increase. A supplied sparse example has 260 rather than 3013 tuples after giving the full-block comparison zero removal and valuation normalization. These are archive-reported local inventory counts, not newly reproduced SAT timings. The theorem is not asserted for arbitrary matrices over a nilpotent coefficient extension.

That packet also supplies mixed-radix Gray updates for affine coefficient extraction, antipodal centered sampling, and projective roots-of-unity extraction. Their gains depend on the exponent profile, available roots, and required precision. In particular halving the sample count can require a larger precision; it is not automatically a wall-time improvement. These kernels can be retained where their hypotheses hold, without multiplying unrelated timing ratios.

The late frontier packet's precision peeling factors guaranteed valuation V from a mode contribution and computes the residual at $P - V$ bits. Divisions use sufficient input precision before reduction, and retained tensors receive new frozen caps; nested caps flatten to the minimum. Exact-weight arrays in the cached algorithm have one common residual ring. This is an inherited compatible optimization, not a new claim of this report.

Finally, the contraction-tree packet replaces flat block construction by binary subset merges and transports a field matrix-multiplication algorithm with exponent below $12/5$ to the actual ring through integer module coordinates and CRT. Its published theorem charges child oracles and construction work. The included small finite matrix checks are not an implementation of the asymptotically fast backend. Our final composition must preserve that limitation and pay for all merges and recursive calls.

4 Normalized contraction interface and inherited lemmas

Let s be an upper bound on the root number of Boolean wires and let $\mathcal{L}$ denote the original description-length parameter. Write $\lambda = \lceil \log_2 s \rceil$. The generic interface assumes maximum root degree at most $\left\lceil s^{9/10} \right\rceil$, polynomial-size descriptions, and initial precision and truncation bounded by a fixed constant times s. Original entry access costs at most $\mathrm{poly}(s + \mathcal{L}) 2^{O(s^{9/10})}$. Tensor representatives retain immutable creation-precision tags rather than being reinterpreted at a later lower modulus.

A recursive node with M wires chooses alphabet r, packet exponent a, retained range u, packet length $L = 2^a$, and $Q = r^a$ pair labels on each side. At retained valuation $i < u$, write $D_i = 2^{a-i}$. The unchanged algebra supplies:

1. a child graph with $(2/r + o(1))M$ wires when padding is asymptotically lossless;
2. at most $(2(1 - 1/r) + o(1))M/D_i$ mode families at level i;
3. sufficient child precision $(6 + o(1))MD_i/u$ when inherited precision and secondary evaluation losses are negligible;
4. full-orientation integral mode weights with leading slope four and the same orientation and residue-family multiplicity as the previous decomposition.

The deformation identity, exact interpolation and recovery, integrality, tensor entry bounds, and recursive-factorization identities behind this interface are inherited assumptions. The new scheduling theorem is a conditional improvement of that interface, not an independent rederivation of those identities.

4.1 Asymmetric deterministic packing

Lemma 4.1 (Inherited asymmetric pair-load bound)**.** *For a bipartite multigraph with M edges, maximum row degree R, maximum column degree C, integer $Q \geq 2$, and integer $h \geq 2$, a polynomial-time exact-rational construction labels vertices on both sides by Q labels so that each pair load obeys*

$$X_{ab} \leq \left(1 + \frac{2}{h-1}\right)\left(\frac{M}{Q^2} + \frac{R}{Q}\right) + 2(hC + 1)\ln Q.$$

Parallel edges are permitted and whole vertices, not individual parallel edges, receive labels.

For context, first greedily place row vertices by degree. Each row-group load is at most $M/Q + R$. Then let $q = 1 + 1/(hC)$ and derandomize the labels of whole column vertices using the exact conditional expectation of $\sum_{a,b} q^{X_{ab}}$. The facts

$$q^w - 1 \leq \frac{w}{C}(q^C - 1), \qquad \frac{1}{\ln q} \leq hC + 1, \qquad \frac{q^C - 1}{C \ln q} \leq 1 + \frac{2}{h-1}$$

give the stated load bound. The independence is across distinct column vertices; no independence of parallel edges is assumed. All conditional expectations are rationals of polynomial bit length in the explicit edge-size model.

For $h \asymp \log s$, the common-pair padding P satisfies $P = (1 + o(1))M$ whenever

$$\frac{QR}{M} = o(1), \qquad \frac{Q^2 C (\log s)^2}{M} = o(1), \qquad \frac{LQ^2}{M} = o(1). \tag{5}$$

The last condition pays for packet-length rounding. At the root both degrees are at most $\Delta = \left\lceil s^{9/10} \right\rceil$. Below the root the retained side has degree at most Δ, but the fresh side has degree at most $2L_{\text{previous}}$. The underlying child construction places retained tensors from both old sides on one new side and new packet tensors on the other; this asymmetry is structural, not a relabeling convenience that can be assumed without proof.

4.2 Joint modes and local entropy

The joint basis uses weights $a + 1 - i + 4q - 2s_2(q)$, where $s_2(q)$ is the binary digit sum. The full source form belongs to a single source group and all its targets belong to the opposite group. Thus it preserves the child-factorization interface. The weighted partition product has leading free energy $\pi^2/(24\theta)$. Combining the family count with the exact taper gives the local bound

$$\log_2 N_i \le (\kappa(r) + o(1))\frac{M}{\sqrt{u}}, \qquad \kappa(r) = \frac{\pi\sqrt{2(1-1/r)}}{\ln 2}. \tag{6}$$

Annulus-sensitive weights can be useful for finite levels, but their uniform slope tends to three. They do not furnish an independent multiplicative improvement on top of the already stronger full-block slope-four basis.

4.3 Streamed separators and paid child evaluations

The closing adaptive-contraction packet supplies a stronger polynomial-space summation than enumerating every global mode tuple. Fix an exact pair-weight array and the Cartesian product of its nonempty pair lists $\mathcal{I}_e$. The group function Z_g depends only on pair indices incident to g. For a group set B, split B into nearly equal parts B_1, B_2, condition on all directed pair indices crossing the split, and recursively sum each side's internal indices. Ancestor-cut assignments remain fixed parameters. Finite distributivity proves that multiplying the two conditional sums and summing over the cut gives exactly the original mode sum.

At a singleton, Z_g is evaluated by the actual prescribed child contraction, not a free oracle. If E_ℓ is the union of cut-pair sets on the path to leaf ℓ, the number of paid child calls is at most

$$\sum_{\ell=1}^{r} \prod_{e \in E_\ell} |\mathcal{I}_e|. \tag{7}$$

The cut sets on one path are disjoint. Sibling-internal domains contribute additive work, not a multiplicative factor to a particular leaf's call count. Internal scalar loops add only a fixed polynomial factor. Index ranks, accumulators, and both recursion stacks have polynomial bit size, so streaming both levels depth first preserves polynomial space.

Let $v(r)$ be the largest number of directed pairs on such a leaf path. Then

$$v(1) = 0, \qquad v(r) = 2\,\lfloor r/2 \rfloor\,\lceil r/2 \rceil + \max\{v(\lfloor r/2 \rfloor), v(\lceil r/2 \rceil)\}. \tag{8}$$

The root cut contributes the first term and the path then enters one child, proving the recurrence. For powers of two, $v(r) = 2(r^2 - 1)/3$. In particular

r	3	4	5	6	8	13	14	15	16
$v(r)$	6	10	18	24	42	118	132	154	170

The joint weighted entropy estimate applied to each fixed path set yields

$$\log_2(\text{paid calls}) \le (\alpha(r) + o(1))\frac{M}{\sqrt{u}}, \qquad \alpha(r) = \frac{\pi\sqrt{2}}{\ln 2}\frac{\sqrt{v(r)}}{r}. \tag{9}$$

This replaces $\kappa(r)$ in the time calculation. The unequal-symbol fraction must not be multiplied in again: it is already embodied by counting only directed pairs with distinct endpoints. This separator identity is inherited from the closing packet; our contribution is the explicit schedule composition and its independently verified strict coefficient comparisons.

5 A streamed polynomial space schedule

Theorem 5.1 (Adaptive alphabet schedule)**.** *Under the interface of Section 4, including its unchanged algebraic identities, there is an explicit deterministic polynomial-space contraction schedule satisfying* (1) *with*

$$C = C_{\text{stream}} = 55.331622120479864\ldots < 55.35.$$

The result holds for every sufficiently large s; finitely many smaller sizes can use direct evaluation. It establishes neither a new logarithmic exponent nor global schedule optimality.

5.1 Rational allocation and integer parameters

For depths $d = 0, \ldots, 6$, use Table 4. Here A_d is the cumulative leading packet budget and $x_d = A_d - A_{d-1}$, with $A_{-1} = 0$. For $d = 7 + j$, $j \ge 0$, put

Depth d	Alphabet r_d	A_d	x_d
0	13	27/2000	27/2000
1	14	21/800	51/4000
2	8	333/10000	141/20000
3	6	1933/50000	67/12500
4	5	861/20000	439/100000
5	4	1183/25000	427/100000
6	4	4999/100000	267/100000

Table 4: A strict interior polynomial-space schedule. The alphabet is allowed to increase from depth zero to one because the root has a stricter two-sided packing constraint.

$$r_d = 3, \qquad x_{7+j} = \frac{1301}{400000}\left(\frac{3}{4}\right)^j. \tag{10}$$

The total leading budget is $\sum_{d\ge 0} x_d = 63/1000$. Define the actual integer parameters by

$$b = \lceil 6\log_2 \lambda \rceil, \qquad u_d = \lfloor \lambda x_d \rfloor, \qquad a_d = \sum_{j=0}^{d}(u_j + b), \qquad L_d = 2^{a_d}, \quad Q_d = r_d^{a_d}.$$

Directly evaluate nodes with $M \le H = \lceil s/\lambda \rceil$. Each child creates a fresh labeling and packet decomposition. Changing its alphabet does not alter an existing parent descriptor or mode basis.

5.2 Packing slack is exact and uniform

The root's principal packing exponent requires

$$\frac{9}{10} + 2A_0 \log_2 r_0 < 1.$$

Below the root the retained-side condition requires

$$\frac{9}{10} + A_d \log_2 r_d < 1.$$

For this schedule, the required strict inequalities have integer witnesses:

$$13^{27} < 2^{100}, \qquad 14^{21} < 2^{80}, \qquad 8^{333} < 2^{1000}, \qquad 6^{1933} < 2^{5000},$$
$$5^{861} < 2^{2000}, \qquad 4^{1183} < 2^{2500}, \qquad 4^{4999} < 2^{10000}, \qquad 3^{63} < 2^{100}.$$

These are exact comparisons, not rounded decimal checks. Every nonroot fresh-side and rounding exponent is bounded by

$$2A_d \log_2 r_d + A_{d-1} < .2 + .063 = .263, \qquad (2 \log_2 r_d + 1)A_d < .263.$$

The corresponding root bounds are smaller. New packet degrees have exponent at most $.063+o(1) < .9$, so the retained-side degree promise is preserved. Fixed slack absorbs the ceiling in Δ, factors polynomial in λ, the additive budget from b, and the active-node bound $s/M \le \lambda$.

5.3 A noncircular depth argument

A common scheduling error is to assume small depth in order to prove the padding needed to establish small depth. Instead consider hypothetical active nodes at every depth up to

$$D = \left\lceil \log_{10/7} \lambda \right\rceil + 10.$$

The fixed prefix is harmless. In the tail, for every such node,

$$u_d \ge \lambda^{\eta + o(1)}, \qquad \eta = 1 - \frac{\ln(4/3)}{\ln(10/7)} = .1934334683\ldots > 0.$$

Thus all active retained ranges grow uniformly and floors introduce only relative $o(1)$ errors. Equations (5) hold uniformly on this hypothetical range, so padding is $1 + o(1)$. Since all alphabets are at least three, every child then has at most

$$(2/3 + o(1))M \le .7M$$

for sufficiently large s. An active path cannot actually reach depth D, since its mass would fall below s/λ. This closes the bootstrap without presupposing the actual depth.

With $h = \lceil \log_2 s \rceil$, the padding error is uniformly $O(1/\lambda) + s^{-\epsilon + o(1)}$ for a fixed $\epsilon > 0$. Its product over $O(\log \lambda)$ levels is $1 + o(1)$. Consequently

$$M_d \le (1 + o(1)) s m_d, \qquad m_0 = 1, \qquad m_{d+1} = \frac{2}{r_d} m_d. \tag{11}$$

In particular $m_7 = 1/5460$ and $m_{7+j} = (2/3)^j/5460$.

5.4 Precision and lower order costs

The inherited precision is $O(s\lambda^2 2^{a_{d-1}})$. At $i < u_d$,

$$D_i = 2^{a_d - i} \geq 2^{a_{d-1}+b+1}.$$

Using $M > s/\lambda$ and $u_d = O(\lambda)$, its ratio to MD_i/u_d is $O(\lambda^4 2^{-b}) = o(1)$. The secondary Hermite evaluation loss is likewise lower order. Hence the exact taper gives the required uniform precision

$$p_t \leq (6 + o(1))\frac{MD_i}{u_d}.$$

Only the finite alphabet set $\{3, 4, 5, 6, 8, 13, 14\}$ occurs; therefore the local entropy errors in (9) are uniform. Polynomial group and interpolation overhead over the entire path adds only $O(\lambda \log \lambda)$ to the logarithm of the running time.

Leaves cost $\exp(O(s/\lambda))$. Original entry routines cost $\exp(O(s^{.9}))$. Packet routines cost $\exp(O(L \log s))$ with $L \leq s^{.063+o(1)}$. All are lower order than $\exp(s/\sqrt{\lambda})$. Arithmetic precisions, labels, interpolation degrees, and immutable descriptions have polynomial bit length. Depth-first evaluation needs polynomial working space.

5.5 Summing and certifying the coefficient

By (9) and (11), the coefficient is

$$C_{\text{stream}} = \sum_{d=0}^{6} \frac{\alpha(r_d)m_d}{\sqrt{x_d}} + \frac{\alpha(3)}{5460\sqrt{1301/400000}\,[1 - 4/(3\sqrt{3})]}. \tag{12}$$

The seven prefix contributions are approximately

$$46.09673615,\ 7.16687600,\ 1.35915226,\ .39277153,\ .15034215,\ .05681105,\ .03592205.$$

The tail contributes $.07301092\ldots$. The tail is a convergent geometric series because $(2/3)/\sqrt{3/4} = 4/(3\sqrt{3}) < 1$.

The checker does not rely on this decimal display. It proves $\pi < 355/113$ and $\ln 2 > 693/1000$ using, respectively, the alternating Machin arctangent identity and the positive atanh(1/3) series. It combines these with rational one-sided square-root bounds and exact integer packing comparisons to obtain

$$C_{\text{stream}} < \frac{1029779221771}{18607088500} = 55.3433828066\ldots < 55.35. \tag{13}$$

The same checker composes the closing separator with the two late variable-alphabet schedules. Their evaluated coefficients are $55.4046659066\ldots$ and $55.4090551361\ldots$, with certified lower enclosures $55.4045521125\ldots$ and $55.4089416944\ldots$. Both exceed 55.4, while our upper enclosure is below 55.35. Thus the strict improvement survives comparison with these stronger compatible compositions, rather than only the supplied fixed-16 value 55.709478.

The proof establishes the all-size asymptotic result; finite checks verify the explicit constants and implementation of the certificate. Neither floating-point optimization nor a finite search establishes global optimality.

6 A refined cached contraction tree schedule

The late contraction-tree packet supplies the strongest explicitly displayed generic cached time certificate in the closing intake, 35.3545880584... with space coefficient 34.2748030050.... The new result in this section is a strict schedule refinement of its binary table construction and CRT backend, not a new matrix multiplication theorem.

Theorem 6.1 (Refined cached tree schedule)**.** *Under the same generic normalized interface and the imported cached-tree/CRT identities, the bounds*

$$T(s,\mathcal{L}) \le \mathrm{poly}(s+\mathcal{L})2^{(C_T+o(1))s/\sqrt{\log_2 s}},$$
$$S(s,\mathcal{L}) \le \mathrm{poly}(s+\mathcal{L})2^{(C_S+o(1))s/\sqrt{\log_2 s}}$$

hold with

$$\begin{aligned} C_T &= \frac{\pi}{\ln 2}\sqrt{\frac{7125}{119}} = 35.0706136705\ldots < 35.071, \\ C_S &= \frac{\pi}{\ln 2}\sqrt{\frac{46875}{833}} = 33.9995016442\ldots < 34. \end{aligned} \tag{14}$$

Both coefficients improve the specified late tree schedule. This is a subexponential-space result, not a polynomial-space theorem or a proof of global optimality over all possible compositions.

6.1 Exact schedule and uniform feasibility

Put $\lambda = \lceil \log_2 s \rceil$ and $b = \lceil 6 \log_2 \lambda \rceil$. The root alphabet is 8 with retained fraction $x = 833/50000$. At depth $d \ge 1$, use alphabet 12 and fraction $y/6^{d-1}$, where $y = 5003/540000$. Thus

$$x + \frac{y}{1-1/6} = \frac{1}{36}.$$

As before u_d is the floor of λ times its fraction, and $a_d = \sum_{j \le d}(u_j + b)$. Here the direct leaf cutoff is *different*: $H = \left\lceil s/\lambda^{3/4} \right\rceil$.

The root symmetric packing exponent is $.9 + 6x = 24999/25000 < 1$. Below the root the retained-side exponent is at most $.9 + \log_2(12)/36 < 3599/3600 < 1$, using $12^{100} < 2^{359}$. Fresh-side and rounding exponents are bounded by $(1 + 2\log_2 12)/36 < 1$; fresh packet degrees have exponent $1/36 + o(1) < .9$. These fixed margins absorb the logarithmic buffers and all polynomial factors in λ.

For hypothetical active tail depths through $(3/4)\log_5 \lambda + O(1)$, retained ranges grow at least as $\lambda^{\eta+o(1)}$, where

$$\eta = 1 - \frac{3\ln 6}{4\ln 5} > 0 \quad \text{because } 6^3 < 5^4.$$

Uniform lossless packing therefore applies. Root shrink is $1/4 + o(1)$; tail shrink is $1/6 + o(1)$ and eventually at most $1/5$, so an active path cannot exceed that hypothetical bound. Floors give uniform relative $o(1)$ errors and accumulated padding remains $1 + o(1)$.

The inherited precision ratio is now $O(\lambda^{15/4}2^{-b}) = o(1)$ because $s/M \le \lambda^{3/4}$. The same exact taper yields $(6 + o(1))MD_i/u$. Leaves cost $2^{O(s/\lambda^{3/4})}$, original entries cost $2^{O(s^{.9})}$, and packet entries cost $\exp(s^{1/36+o(1)})$; all remain lower order. Changing the cutoff is essential to the taper-growth proof and must not be hidden by mechanically copying the s/λ proof.

6.2 Binary tables and the imported matrix backend

For a block split into sizes a, b inside r groups, the matrix dimensions are controlled by directed-pair cardinalities

$$2a(r-a-b), \qquad 2ab, \qquad 2b(r-a-b).$$

Only the A–B indices are shared between factors; the two exterior index collections are disjoint. This is the exact binary table factorization. The inherited weighted entropy converts a fixed field exponent $12/5$ into squared cost $\sum v - (21/25)\min v$ for these three cardinalities. The primary paper *More Asymmetry Yields Faster Matrix Multiplication* supplies a strict exponent below $12/5$; no claim about the current record is required.

The actual ring is not a field. The inherited bridge expands its polynomially many module coordinates into bounded integer matrix products, uses sufficiently many nonexceptional odd primes, reconstructs by CRT, combines structure constants, and reduces to the required power-of-two modulus. Only units in the auxiliary prime fields are inverted. Polynomial coordinate, prime, and bit overheads do not change the leading coefficient. The asymptotically fast backend is an imported theoretical algorithm, not implemented or benchmarked in this bundle.

6.3 The complete recursive maximum

Let $k = \pi\sqrt{2}/\ln 2$, $K_0 = k/\sqrt{x}$, $K_1 = k/(4\sqrt{y})$, and $\gamma = 1/\sqrt{6}$. Tail scale $d \geq 1$ is $K_1\gamma^{d-1}$. The exact pair ledger gives

	cache a_r	tree and triangle b_r	storage j_r
$r = 8$	$\sqrt{14}/8$	$\sqrt{798/25}/8$	$\sqrt{30}/8$
$r = 12$	$\sqrt{22}/12$	$2\sqrt{3}/5$	$2/3$

The largest binary merges have squared cardinalities $766/25$ and $1632/25$, respectively; both are smaller than the final triangle values $798/25$ and $1728/25$. All singleton evaluations are still paid recursive contractions.

Suppressing only polynomial factors, the recurrence is $T_d \leq 2^{a_{r_d}K_d}T_{d+1} + 2^{b_{r_d}K_d}$. Hence

$$C_T = \max\left\{b_8K_0, a_8K_0 + K_1\max\left(b_{12}, \frac{a_{12}}{1-\gamma}\right)\right\}. \tag{15}$$

The checker proves $a_{12}/(1-\gamma) < b_{12}$ and encloses the entire descendant candidate by its evaluated value $34.7601269273\ldots$, strictly below the root's $35.0706136705\ldots$. Thus the root dominance is a result of the full recurrence, not an assumption or a root-local shortcut.

Storage is the maximum, not the sum, of leading exponents:

$$C_S = \max\{j_8K_0, j_{12}K_1\}. \tag{16}$$

Root storage is $33.9995016442\ldots$ and tail storage is $11.0986405851\ldots$. Polynomially many frames, coordinate products, and CRT residues add only polynomial overhead per table entry. Sequential fixed-base matrix multiplication uses quadratic element storage, as required by the imported bridge. No free oracle or unit-cost enormous integer is introduced.

6.4 Comparison and lower memory choices

The exact checker proves every packing guard, budget identity, taper-growth inequality, full-recursion dominance, storage maximum, and strict comparison with the late $35.354588/34.274803$ source. Machin/atanh series and integer square-root enclosures decide all gates in rational arithmetic.

The separately certified lower-memory companion has time coefficient 37.3246112387... and space coefficient 33.6230409217.... It retains the earlier root-7 adaptive schedule, replacing only its evaluator by the late CRT/tree method. The limiting ancestor-cache sum, not the smaller local root cost, determines its time. Its exact factor ledger and dominance checks are in `late_cache_checks/verify_lowmemory_companion.py`. Similarly, the late degree-$s^{1/12}$ adaptive cached option has 15.391933/14.250161 under a narrower premise. The complete choice is a resource-qualified portfolio, not a claim that one number is universally best. Practical kernel improvements below remain useful without requiring a globally optimal high-memory schedule.

7 A degree sensitive streamed schedule

A second polynomial-space schedule is useful when the initial graph has lower degree. Replace the generic root promise by

$$\Delta \le \left\lceil s^{51/100} \right\rceil .$$

Keep the unchanged algebraic interface, creation-precision semantics, and entry access assumptions. In particular original entry routines may still cost $\mathrm{poly}(s+\mathcal{L})2^{O(s^{.9})}$, since this remains lower order.

Theorem 7.1 (Degree sensitive schedule)**.** *Under this narrower degree premise, there is a polynomial-space schedule with coefficient*

$$C_{.51} = 25.16376541522859\ldots < 25.2$$

in (1)*. The exact certificate proves*

$$C_{.51} < \frac{1326135054955358}{52688987071875} = 25.1691127246\ldots < 25.2.$$

This case matters because the supplied high-moment proposal's alternative SAT grouping has initial degree $n^{1/2}\,\mathrm{polylog}\,n$ with wire budget $s = O(n \log\log n)$. Where that imported construction applies, its normalized degree is eventually at most $s^{.51}$. The theorem thus preserves that useful lower-degree route. It does not turn the wire coefficient directly into a SAT-variable coefficient or eliminate the reduction's $\log\log n$ factor.

Use the same λ, b, H, u_d, a_d conventions as before, with the finite prefix

d	r_d	A_d	x_d
0	15	627/10000	627/10000
1	15	2269/20000	203/4000
2	8	357/2500	587/20000
3	6	81/500	12/625
4	5	1771/10000	151/10000
5	4	1911/10000	7/500
6	4	1999/10000	11/1250

For $d = 7+j$, set $r_d = 3$ and $x_d = (391/40000)(3/4)^j$. The total budget is 239/1000 and $m_7 = 1/6750$.

This is not a naive rescaling of the generic theorem. With $\gamma = .51$, the required constraints now include

$$\gamma + 2A_0 \log_2 r_0 < 1, \qquad \gamma + A_d \log_2 r_d < 1,$$
$$(1 + 2\log_2 r_d)A_d < 1, \qquad 2A_d \log_2 r_d + A_{d-1} < 1.$$

Packet rounding is now a binding restriction. The last condition follows from the preceding one and $A_{d-1} \le A_d$. All new packet degrees are $s^{.239+o(1)} < s^{.51}$, preserving the retained-side invariant.

The exact guards include $15^{627} < 2^{2450}$ for the root degree, $15^{4538} < 2^{17731}$ for depth-one rounding, and $3^{478} < 2^{761}$ for tail rounding. The root degree and depth-one rounding gaps are each about $7.6 \cdot 10^{-5}$, strictly positive rather than rounded equalities. They absorb the logarithmic buffers and active-node factor $s/M \le \lambda$. The same depth bootstrap and precision proof apply. Packet entry work is at most $\exp(s^{.239+o(1)})$, still lower order; descriptions and arithmetic remain polynomial-size.

The coefficient is the analogue of (12), with the displayed prefix and tail term

$$\frac{\alpha(3)}{6750\sqrt{391/40000}\,[1-4/(3\sqrt{3})]}.$$

The separately written exact checker certifies all degree and rounding inequalities and the rational upper bound in Theorem 7.1.

The local factor is now $\alpha(r)$ from the inherited streamed evaluator, not the older tuple-enumeration $\kappa(r)$. The latter would give the valid but weaker intermediate value 29.353412. No degree promise, rounding budget, precision condition, or child-factorization interface is changed by the streamed sum. The narrower degree-$s^{1/12}$ branches remain separate portfolio choices. For example, the inherited constant-48 schedule composed with the same separator has coefficient 22.1981057366 . . . in polynomial space: here $v(48) = 1536$ and its value is

$$\frac{\pi\sqrt{2}}{\ln 2}\,\frac{\sqrt{1536}}{48}\,\frac{\sqrt{14}}{1-\sqrt{8}/24}.$$

The late cached specialization has coefficient 15.391933 with additional memory. Neither uses the same degree premise as the new .51 theorem. No direct SAT-variable coefficient or new density compression is inferred.

8 Sparse exact conditional expectation updates

The initial cached asymmetric packer has the repeated candidate-score bottleneck described below. The closing frontier packet already supplies a dense quadratic update, so only the additional positive-support sparsity refinement is new relative to the full reviewed corpus; the timings compare against that stronger dense-delta formula as well as the historical cubic one. After deterministic row grouping let w_{aj} count edges from row group a to column vertex j, and let ℓ_{ab} be current pair loads. Set $q = 1 + 1/(hC) > 1$, with $C = 1$ for an empty graph. The incoming code already caches powers $q^0, \dots, q^M$ and future row products.

8.1 The candidate delta identity

Immediately before assigning column j, remove its factor from each future product and call the result F'_a. If j is assigned label b, its exact conditional expectation is

$$E_b = \sum_a F'_a \sum_c q^{\ell_{ac}+\mathbf{1}_{c=b}w_{aj}}. \tag{17}$$

Define

$$U_a = F'_a(q^{w_{aj}} - 1), \qquad \delta_b = \sum_a U_a(q^{\ell_{ab}} - 1).$$

Separating the candidate-independent part of (17) gives

$$E_b = B + \sum_a U_a + \delta_b, \qquad B = \sum_a F'_a \sum_c q^{\ell_{ac}}.$$

If P is the current conditional expectation before choosing a label, the law of total expectation says $P = Q^{-1} \sum_b E_b$. Thus

$$E_b = P + \delta_b - \frac{1}{Q} \sum_c \delta_c. \tag{18}$$

This is an exact rational identity, not a floating-point approximation or a surrogate objective.

Proposition 8.1 (Exact sparse replacement)**.** *The full candidate evaluator and the sparse-delta evaluator choose identical labels under lexicographically first minimizer tie-breaking. They produce identical pair loads and conditional potentials at every step. If requested, all full candidate certificates can be reconstructed from* (18)*.*

Proof. A summand in δ_b is nonzero exactly when both $w_{aj} > 0$ and $\ell_{ab} > 0$. On this support it is strictly positive because $q > 1$ and $F'_a > 0$. Thus the score dictionary need contain only labels reached through active rows and currently positive cells. A missing label has score zero and is a minimizer. If any label is absent, choose the least absent label; otherwise choose the least label among the stored minimum scores. This is precisely the full evaluator's minimizer and tie rule, since (18) differs from δ_b by a candidate-independent constant. Update the chosen loads and potential using the same identity. Induction from the identical initial state proves all asserted equalities. □

The proof also covers empty columns and empty graphs. In particular, absent dictionary entries mean exact zero, not missing numerical data. Equality checks explicitly exercise such ties; tests that merely compare final maximum load would be too weak.

8.2 Work and bit complexity

Let $A_j = \{a : w_{aj} > 0\}$ and let $S_a(j)$ be the positive support of row a in the assigned load table before column j. Sparse score accumulation takes

$$O\left(\sum_j \sum_{a \in A_j} |S_a(j)| \right) \tag{19}$$

exact rational score-accumulation operations. Building active-row coefficients costs an additional $O(W)$ operations, and empty-column bookkeeping or minimum selection contributes $O(N_{\mathrm{col}})$ beyond the support scan. If $W = \sum_j |A_j|$ is the number of nonzero aggregated row-column incidences, this is at most $O(WQ)$, and at most $O(N_{\mathrm{col}} Q^2)$.

In contrast, the delivered cached asymmetric scorer examines Q candidates, each with a Q-by-Q sum, giving $\Theta(N_{\mathrm{col}} Q^3)$ additions/lookups and $\Theta(N_{\mathrm{col}} Q^2)$ multiplications in that scoring stage. A dense-delta evaluator already reduces this to $O(WQ)$, as anticipated by the closing packet's quadratic scorer; sparse deltas can then improve on that stronger reference through (19). We benchmark all three, so the advertised gain is not solely against the weakest version.

These counts do not make exact rational operations unit-cost in bit complexity. The implementation also reads the input, groups rows with a heap, caches q^0 through q^M, initializes future factors, copies $O(N_{\mathrm{col}} Q)$ list entries, and produces Q^2 dense output cells. These shared costs are included in timings. Here M is total edge multiplicity; a length-M powers cache is pseudo-polynomial if multiplicities

are supplied in a succinct binary encoding. The contraction interface measures input size by explicit wires, where that issue is already accounted for.

Complete per-step candidate certificates require $O(N_{\mathrm{col}}Q)$ extra exact outputs. They are enabled in equivalence checks and disabled equally in all timed methods. The code exposes optional invariants and counters; both are disabled equally in timing calls and independently checked in untimed runs.

9 Cauchy recurrence reproduction and new ratio fusion

The initial joint-Cauchy implementation computes the bth feature by traversing all $h < b$ anew, for each evaluation row and each mode. Full feature tables therefore take cubic unit-ratio arithmetic. The implementation below changes this historical full-table workload without changing any feature or assuming nonunit division is legal modulo a power of two. The closing packets already contain this recurrence and its valuation-safe modular realization; this section records our independent reproduction, not final-corpus novelty. Section 9.4 compares the further refinement against the quadratic recurrence rather than against products.

Precedent and novelty boundary. The earlier `cauchy_mode_improvement` report already suggests incrementally updating its different factorial-mode basis, including a relation of the form $A_{b+1}(z) = A_b(z)(z-2b)/(1-bz)$. We therefore do not claim to invent incremental Cauchy evaluation or to be first to obtain quadratic tables for every basis. The reproduced joint-mode implementation uses safe valuations and shared batch inverses; only the additional fusion in Section 9.4 is claimed as a new refinement relative to the closing corpus.

9.1 Defining products and recurrence

Let $N = L/2$, $R = 2^i$, and

$$s_j = (j \bmod R) + 2R\lfloor j/R \rfloor, \qquad E_b = 2\lfloor b/R \rfloor - s_2(\lfloor b/R \rfloor).$$

At a retained interpolation node, $T = t + 2L\zeta$ with $v_2(t) = i$ and $-L \le t < L$. Write $\mathcal{C}(T) = \prod_{v=-L}^{L-1}(T-v)$ and $\Gamma(T) = \mathcal{C}(T)/\mathcal{C}'(0)$. The source kernel is the cancelled polynomial $\Gamma(T)/(T+x-y)$. The two product features are

$$f_b(x) = \frac{T}{T+x-s_b}\prod_{h<b}\frac{x-s_h}{T+x-s_h}, \tag{20}$$

$$g_b(y) = \frac{T}{T+s_b-y}\prod_{h<b}\frac{s_h-y}{T+s_h-y}. \tag{21}$$

Cancellation between successive indices proves

$$f_{b+1}(x) = f_b(x)\frac{x-s_b}{T+x-s_{b+1}}, \tag{22}$$

$$g_{b+1}(y) = g_b(y)\frac{s_b-y}{T+s_{b+1}-y}. \tag{23}$$

At node j, only $b \le j$ need be emitted; subsequent entries vanish. The integral normalized factors are $F_b = 2^{-E_b}f_b$ and $G_b = 2^{-E_b}g_b$. The actual target factor includes the common prefactor and scale

$$\frac{\Gamma(T)}{T}\,2^{2E_b}F_b.$$

The product reference used in the comparisons also skips the known triangular zeros. Thus the improvement does not rely on performing avoidable products for entries known to vanish.

9.2 Valuation safe modular arithmetic

A naive implementation of (22) may try to invert an even residue or may shift a reduced integer to the right, losing necessary precision. The implementation instead represents each nonzero unnormalized feature by a pair (v, u) with integer valuation v and a unit residue u modulo 2^p. For every numerator and denominator, extract its *known exact* power of two before reducing the unit part. Update v by the difference of valuations and multiply u by the unit ratio. Emit

$$2^{v-E_b}u \pmod{2^p}.$$

The inherited valuation lemma guarantees $v - E_b \geq 0$ for emitted normalized features. A negative valuation in an individual ratio is harmless: it changes the separate integer account, not the legality of an inversion in the residue ring. Exact zero factors and zero prefactors are handled explicitly.

The test ring is the genuine degree-two unramified extension

$$(\mathbb{Z}/2^p\mathbb{Z})[U]/(U^2 + U + 1).$$

An element with nonzero residue modulo 2 is a unit. This is not a prime field, and the test harness includes non-prime residues and explicit rejection of nonunits. The general batch-inversion argument is valid in any commutative ring for unit inputs, including the scalar units occurring alongside nilpotent tensor coordinates; the finite implementation tests do not claim coverage of every possible extension degree.

9.3 Historical batch inverse setup

All denominators have the form $T + d$. Their distinct differences belong to

$$\{2Rq + r : -(N/R - 1) \leq q \leq N/R - 1,\ -(R - 1) \leq r \leq R - 1\},$$

whose cardinality is

$$K = (2N/R - 1)(2R - 1) < 4N = 2L.$$

Extract and cache each denominator unit once. Prefix products followed by one inversion and a reverse pass compute all inverse units in $3K$ ring multiplications under the code's counting convention. The argument uses only multiplication and inversion of a unit product, so it does not depend on field division.

Both implementations share this batch setup, use identical modular arithmetic, and include initialization and complete dense output in their operation counts and timings. This baseline is stronger than repeatedly invoking the incoming product helper with uncached inverses.

Proposition 9.1 (Historical full table arithmetic)**.** *Let I be the common initialization multiplication count. For the original two-inversion prototypes and nonzero-prefactor cases, $I = 3K + 4L - 1$. Including target scaling, the product and recurrent algorithms use exactly*

$$M_{\text{product}} = I + \frac{N(N+1)(2N+1)}{3} + \frac{N(N+1)}{2},$$
$$M_{\text{recurrent}} = I + 2N^2 + \frac{N(N+1)}{2}$$

ring multiplications. Both use two unit inversions including the prefactor, or fewer in the zero-prefactor case. Complete table work decreases from $\Theta(N^3)$ to $\Theta(N^2)$ arithmetic operations.

Proof. At evaluation node j, the two defining-product families use $2\sum_{b=0}^{j}(1+2b) = 2(j+1)^2$ multiplications. Summing over $j = 0, \ldots, N-1$ gives $N(N+1)(2N+1)/3$. The recurrent families instead use $2(1+2j)$ multiplications at node j, whose total is $2N^2$. Target scaling contributes the same triangular term $N(N+1)/2$ in both implementations. The shared initialization contributes I. The exact operation-counter checks verify these conventions, including initialization, rather than counting only the new inner loop. □

Writing all dense ring entries itself costs $\Omega(N^2)$, so the recurrent algorithm is optimal up to constants for this output requirement. Dense storage is $\Theta(N^2)$; streaming columns needs $O(L+N)$ working ring elements excluding output. These are component bounds. They do not reduce the proven SAT leading exponent coefficient, because feature preparation was already polynomial or lower order in that analysis.

For fixed small precision, high modes can vanish before the full index range. For a partial or on-demand query, most features may never be needed. The full-table result is not a universal claim about those workloads; they require their own precision-aware implementation and benchmark.

9.4 New ratio fusion within the quadratic recurrence

The closing packet already supplies the recurrence and a one-inversion setup. The surviving refinement is to precompute each normalized transition ratio and to absorb the common target prefactor once. Our comparator is an independently authored, operation-matched reproduction of that late algorithm, not an execution of its archive code. Both variants use the same modular ring backend, fresh preparation, triangular skipping, complete dense outputs, and one unit inversion. Power-of-two masking in the shared backend is also credited as a prior optimization.

Consecutive pivots have gaps $g_b = s_{b+1} - s_b \in \{1, R+1\}$; for $R = 1$ only the gap 2 occurs. At a real slot $x = s_j$ and transition $b < j$, put $d = x - s_{b+1} \geq 0$. The two transition ratios are

$$\frac{d+g_b}{T+d}, \qquad \frac{-d-g_b}{T-d}.$$

For each nonnegative slot difference and possible gap, cache the exact valuation difference and the quotient of unit parts modulo 2^p. The signed difference set has $K < 2L$ elements, so this uses $O(L)$ entries. Each later transition adds the cached integer valuation and performs *one* ring multiplication, instead of separately multiplying the numerator unit and inverse denominator.

Write the nonzero common prefactor $\Gamma(T)/T = 2^h\rho$, with ρ a unit residue and $h \geq a+1-i \geq 0$ by the inherited prefactor valuation. Form the target starting seed ρT_{unit} once. Target states now contain ρ, and their emitted value is

$$2^{h+v_f+E_b}\,\text{state};$$

source output is still $2^{v_g-E_b}\,\text{state}$. Here v_f, v_g are the exact unnormalized feature valuations. The inherited integral-feature lemma ensures $v_f, v_g \geq E_b$, so no emitted value requires a negative shift. Zero prefactors produce zero target states without attempting to invert them. The valuation account is not reconstructed from a reduced zero residue; this preserves cases where a displayed source output is zero but a later normalized output becomes nonzero.

Proposition 9.2 (Fused table work)**.** *Let $g = 1$ for $R = 1$ and $g = 2$ otherwise. With common initialization cost I, the strengthened quadratic baseline and the fused implementation have counts*

$$M_{\text{baseline}} = I + (5N^2+N)/2,$$
$$M_{\text{fused}} = I + g(K+1) + 1 + N^2 + N.$$

Both use one unit inversion. For the benchmark's nonzero extension prefactor, $I = 3K + 2L + 3$. Both store $O(N^2 + L)$ ring elements including output.

Proof. The implementation prepares $g(K + 1)$ possible transition ratios, a safe linear-size envelope including some unused pairs, and uses one multiplication for the common target seed. There are $N(N - 1)$ transitions across the two triangular tables and $2N$ initial states. After fusion each costs one multiplication, giving $N^2 + N$. The unfused recurrence uses two multiplications per transition and a further triangular target-scaling pass, giving $(5N^2 + N)/2$. Prefactor-denominator inversion is included in the shared batch, so neither algorithm needs a second inversion. □

The leading ratio of these abstract ring-multiplication counts tends to 5/2. It is *not* a 2.5-fold wall-time guarantee: scalar-times-ring and general ring multiplications have different implementation costs, and the new ratio cache adds preparation. Both algorithms remain quadratic; small tables may lose. The inspected late code has neither this transition-ratio cache nor a globally absorbed target prefactor. Novelty is scoped to that inspected report and code, not to all classical common-subexpression elimination or numerical software.

10 Finite evidence and measured performance

Proofs and finite experiments answer different questions. The schedule argument proves a conditional asymptotic statement for all sufficiently large inputs. Exact checkers certify its specified constants and exercise concrete kernels. Timings measure the independently written code on selected local fixtures. None is interchangeable with a complete SAT solver or a proof-assistant verification.

10.1 Correctness and independent audit

The packing checker compares the cubic formula, dense deltas, and sparse deltas on 688 graph cases and 32 further random sparse cases; it checks 1,352 identical step certificates and explicit direct-completion cases. Seven negative controls test rejection behavior. It passes under both ordinary Python and `python -O`; correctness is not encoded solely in removable assertions.

The Cauchy checker uses a separate exact rational oracle over $\mathbb{Q}(U)$, not merely a second path through the same modular implementation. Its recorded checks include 120 rational configurations, 480 modular configurations, 30,048 feature equalities, 15,024 modular reconstructed kernel entries, 3,756 exact kernel entries, 960 operation-count equalities, 20 zero-prefactor configurations, 11 larger cases, and five invalid inputs.

The new fused checker compares both one-inversion variants against the earlier independent implementation and the exact $\mathbb{Q}(U)$ oracle. It records 720 configurations, 152,928 table-entry comparisons, 30,048 exact rational checks, and an explicit zero-output-then-unit regression. Both normal and optimized modes pass; per-table multiplication counts and one inversion are checked rather than inferred from timings.

An independent audit adds 760 three-method packing configurations, 90 direct-completion configurations, and 2,280 controls with instrumentation disabled. It checks 10,850 exact Cauchy reconstruction entries, 86,800 modular feature equalities, 208 zero-prefactor controls, 2,080 operation-count controls, 224 extension-unit inversions, and 16 nonunit rejections. Both normal and optimized Python modes pass. The schedule replay harness additionally includes deliberately false constant, path-profile, and logarithmic-guard controls in ordinary and optimized modes; failed certificates raise explicit exceptions

rather than relying on removable assertions. These are finite component checks, not exhaustive correctness over every graph, precision, or ring extension.

10.2 Timing protocol

All reported timing runs use Python 3.12.14 on Linux x86-64 with glibc 2.41. Each method receives a checked warmup, then five samples. Method order is rotated or alternated to reduce order bias. Reported values are medians, with all raw wall-clock and process-CPU samples preserved. Input generation is excluded; common preprocessing, initialization, and dense output are included. Equality is checked outside the measured region. Optional counters and invariants are disabled equally during timing, with counts taken in separate checked runs.

This is a shared execution host, not a dedicated benchmark machine. Very small measurements show substantial scheduling and interpreter noise, and no confidence interval or universal speedup is inferred from five samples. Process-CPU measurements help identify but do not eliminate environmental variability. First-pass results are retained separately rather than replaced without provenance.

10.3 Packing kernel measurements

The final run contains eight fixed-shape 10-by-12 fixtures and three larger sparse fixtures with $2Q$ vertices on each side and four unit edges per row. The latter keep the number of labels below the vertex count on both sides. Table 5 shows all eleven records. Times are wall-clock medians in milliseconds; C/S and D/S compare the cubic and dense-delta methods against sparse deltas.

Family	Q	Density	Cubic	Dense	Sparse	C/S	D/S
Fixed	4	0.120	1.234	0.498	0.409	3.02	1.22
Fixed	4	0.650	2.180	1.772	1.836	1.19	0.96
Fixed	8	0.120	6.043	0.639	0.456	13.25	1.40
Fixed	8	0.650	15.098	2.828	3.703	4.08	0.76
Fixed	16	0.120	111.849	1.681	0.629	177.74	2.67
Fixed	16	0.650	67.158	9.066	4.210	15.95	2.15
Fixed	24	0.120	146.313	1.784	0.993	147.35	1.80
Fixed	24	0.650	168.805	15.978	4.707	35.87	3.39
Four per row	8	0.250	25.327	4.541	2.967	8.54	1.53
Four per row	16	0.125	144.354	8.981	3.630	39.77	2.47
Four per row	24	0.083	685.391	18.581	6.795	100.87	2.73

Table 5: Packing kernel wall-time medians in milliseconds, five samples per method. Ratios below one are regressions, not omitted results.

For the four-per-row fixtures at $Q = 8, 16, 24$, sparse deltas reduce wall time by factors 8.54, 39.77, and 100.87 relative to full candidate scoring; relative to the stronger dense-delta reference the factors are 1.53, 2.47, and 2.73. CPU ratios are closely comparable. Deterministic sparse score-term counts are 159, 341, and 577, compared with dense-delta counts 432, 1792, and 4272. These counts isolate the support saving without mistaking shared initialization for free work.

The dense-delta column matters: it separates the algebraic removal of a repeated full score from the additional gain due to occupied-cell sparsity. Small or dense fixtures can erase the sparse dictionary advantage, and at least one recorded case has sparse deltas slower than dense deltas. Neither the proof nor the data warrants selecting sparse dictionaries unconditionally for every platform and workload.

A production implementation could dispatch between dense and sparse delta accumulators using observed support, while preserving the same exact formula.

The historical cubic reference is an independently authored implementation of the initial cached scorer, with common heap grouping and preprocessing. The dense-delta reference already implements the stronger algebraic saving found in the closing packet and additionally skips inactive aggregated rows. It is the main comparator for the surviving sparse-support claim. It is not a timing of the incoming archive script itself. The speed ratios are therefore comparisons between the delivered implementations of those formulas, not claims about every possible implementation of the original report.

10.4 Historical recurrence measurements

The original twenty-record product-versus-recurrence experiment is preserved in `results/cauchy_benchmark_confirm.json`, including small-case regressions. After the closing review it is classified as independent reproduction: the quadratic recurrence already exists in the late corpus. Its 27–32 ratios at $N = 128$ are not improvements over that newer algorithm. The fused comparison reported below is the evidence for the surviving new Cauchy refinement.

10.5 Fused recurrence against the matched quadratic baseline

The final comparison uses the shared power-of-two `MaskRing` backend in both algorithms, matching the late implementation's reduction optimization. Both include complete fresh setup, both dense tables, known triangular zeros, and one unit inversion. The baseline is an independently authored mathematical reproduction, not the archive executable. Every timed output was checked outside the clock. Five paired alternating-order samples and symmetric warmups were used.

The final record concatenates disjoint small and large fixture runs without changing their raw samples; their source-file hashes are included in the JSON. Table 6 shows all 23 records. Precision is $p = 4N + 16$, $\zeta = U$, and times are wall-clock medians in milliseconds. The multiplication ratio includes preparation, not just table loops.

Table 6: Matched one-inversion quadratic baseline versus fused transitions.

N	i	p	Baseline	Fused	Wall ratio	CPU ratio	Mult. ratio
2	0	24	0.162	0.172	0.94	0.94	1.00
2	1	24	0.091	0.206	0.44	0.43	0.89
4	0	32	0.188	0.145	1.29	1.30	1.19
4	1	32	0.277	0.162	1.71	1.73	1.01
4	2	32	0.257	0.148	1.74	1.58	1.06
8	0	48	0.377	0.355	1.06	1.06	1.44
8	2	48	0.633	0.448	1.41	1.42	1.22
8	3	48	0.444	0.454	0.98	0.98	1.32
16	0	80	1.103	0.975	1.13	1.13	1.74
16	2	80	1.047	1.452	0.72	0.72	1.47
16	4	80	1.064	0.978	1.09	1.09	1.63
32	0	144	3.631	2.299	1.58	1.58	2.01
32	3	144	3.532	2.777	1.27	1.27	1.76
32	5	144	3.565	2.171	1.64	1.64	1.92
64	0	272	23.861	13.810	1.73	1.66	2.21
64	3	272	20.783	13.460	1.54	1.58	2.02

N	i	p	Baseline	Fused	Wall ratio	CPU ratio	Mult. ratio
64	6	272	47.541	21.201	2.24	2.24	2.16
128	0	528	168.492	93.267	1.81	1.81	2.34
128	4	528	203.468	113.722	1.79	1.79	2.22
128	7	528	313.380	102.629	3.05	3.07	2.31
256	0	1040	2736.851	1587.452	1.72	1.72	2.42
256	4	1040	2660.738	1160.108	2.29	2.30	2.34
256	8	1040	1043.590	716.260	1.46	1.43	2.40

At $N = 256$, $p = 1040$, the three grid choices give baseline/fused wall-time ratios 1.72, 2.29, and 1.46, with CPU ratios 1.72, 2.30, and 1.43. The corresponding total multiplication ratios are about 2.34–2.42. They are different quantities: the baseline's scalar multiplications can be cheaper than the fused algorithm's general ring multiplications.

Across all 23 final records the wall ratio ranges from about .44 to 3.05; values below one are retained regressions. The original percent-reduction backend experiments and a confirmatory run are also preserved, including additional regressions. They are not pooled with the mask-matched run because the backend differs. The final backend was chosen to match an optimization already present in the late code, not to imply that every earlier run improved. Shared-host noise, precision, grid choice, and preprocessing prevent a universal runtime claim. These are full-table component experiments, not SAT timings or partial/on-demand feature benchmarks.

11 Reproducibility and artifact map

The bundle separates source evidence, mathematical derivations, executable kernels, and recorded observations. The initial archive hashes are fixed in `incoming/manifest.json`, and the closing pin and five further identities in `late_incoming/manifest.json`. The report source is `report.tex`; the compiled `report.pdf` is rendered and visually checked before delivery. A bundle-level manifest records the final files, so later modifications can be detected. Selected primary TeX and README snapshots are identified by `sources/manifest.json`; they complement the incoming archive identities.

Run checks from the bundle root with Python 3.12 or a compatible Python version. The exact programs use standard-library arithmetic rather than an external symbolic dependency. Representative commands are:

```
python3 asym_schedule_checks/verify_all_certificates.py
python3 asym_schedule_checks/verify_streamed_composition.py
python3 late_cache_checks/verify_late_cache.py
python3 late_cache_checks/verify_lowmemory_companion.py
python3 code/check_balancing.py
python3 -O code/check_balancing.py
python3 code/verify_fused_cauchy.py
python3 qa/performance_audit_checks.py
python3 -O qa/performance_audit_checks.py
```

For the full five-sample experiments, use:

```
python3 code/benchmark_balancing.py results/balancing_replay.json 5
python3 code/benchmark_fused_cauchy.py --max-a 9 --repeats 5
```

The Cauchy command writes its standard benchmark filename unless `-output` is supplied; use a fresh copy of the bundle if the delivered historical observations should remain untouched. Repeated timing

output is expected to differ across machines. Exact output equality and deterministic arithmetic counts, not historical wall times, are the principal portable checks.

11.1 How to reproduce a fair performance comparison

Do not compare an initialized recurrence against an uninitialized product program, or a sparse evaluator with checks disabled against a full evaluator with invariants enabled. Use the shared ring, initialization, input, output shape, tie policy, and instrumentation settings in the delivered harnesses. Keep input generation outside both measured regions. Retain all methods' raw samples and check equality before interpreting speed ratios.

For Cauchy tables, match N, retained valuation i, precision p, extension ring, zero treatment, target scale, and dense versus streamed output. For packing, match exact q, row-grouping order, multiedge counts, certificates, and output format. An implementation change that alters any of these may still be useful, but it is a different experiment and must be labeled accordingly.

File or directory	Role
`streamed_composition_new.md`	Final streamed schedule and strong comparisons
`asym_schedule_checks/verify_streamed_composition.py`	Final polynomial-space coefficient certificate
`late_cache_checks/verify_late_cache.py`	Final CRT/tree schedule and resource certificate
`asym_schedule_checks/verify_degree_51_schedule.py`	Degree-sensitive packing and coefficient certificate
`sparse_delta_proof.md`	Candidate identity, ties, and arithmetic scope
`cauchy_review.md`	Source review, recurrence, and memory comparison
`packing_review.md`	Three packing reviews and seven-arrival synthesis
`code/sparse_balancing.py`	Three exact packer implementations
`code/fused_cauchy.py`	Shared modular ring and matched quadratic kernels
`code/check_balancing.py` and `code/verify_fused_cauchy.py`	Self-authored finite correctness checks
`code/benchmark_balancing.py` and `code/benchmark_fused_cauchy.py`	Reproducible timing harnesses
`qa/performance_audit_checks.py`	Independent component and record audit
`results/`	Raw samples, exact counters, and machine-readable outcomes

Table 7: The report summarizes the evidence; the executable and raw files preserve the reproducible detail.

12 Limitations and next research decisions

The present work supports concrete improvements without claiming to settle the wider conjectural program. The strongest claims have the following boundaries.

1. **Conditional upstream correctness.** Deformation, integrality, recovery, entry access, and recursive tensor factorization remain inherited. Static review and component checks are not a full independent proof of these foundations.
2. **No new logarithmic order.** The known normalized $s/\sqrt{\log s}$ scale and density envelope remain. The proven schedule gain is in a leading coefficient.

3. **No all-algorithm optimum.** Numerical schedule search is a discovery aid. The explicit rational construction is certified, but no proof excludes better alphabet schedules, mode bases, matrix methods, or different algorithms.
4. **No memory conflation.** Polynomial-space and cached extra-space frontiers are different resources. Specialized low-degree bounds have stronger premises. The independent Claude method lacks directly comparable constants.
5. **No end-to-end timing.** The tests measure packing and full Cauchy tables, not SAT solving or complete tensor contraction. The asymptotic leading coefficient and measured component ratios must not be multiplied into a hypothetical SAT runtime.
6. **Finite precision and workload matter.** Dense tables at growing precision are not partial feature queries or low-precision truncated contractions. Sparse score support is not guaranteed on every input.

The most useful next implementation step is an end-to-end contraction harness with exact resource accounting, not a larger isolated headline timing. It should preserve creation precision, use the strongest applicable degree and memory regime, dispatch between sparse and dense delta scoring, and request only Cauchy features actually needed by a precision-aware contraction. Such a harness would reveal whether these kernels matter in the total runtime and would test the interfaces still assumed in the theoretical analysis.

A stronger *asymptotic order* would require a genuinely new structural ingredient, such as a proved compact summation or degree-preserving recursive representation. Merely optimizing the constants already present cannot establish that claim. The current report therefore delivers a narrower but reproducible result: better certified schedule coefficients on specified resource frontiers and exact component arithmetic reductions with measured, explicitly limited practical benefits.

A Archive identities and source anchors

The initial seven archives below are identified in full by `incoming/manifest.json`, which includes their SHA-256 digests and source blob identifiers. The short names used in the text expand to:

- `eth_asymmetric_packet_balancing_2026-10-26.zip`; theorem labels `thm:pair-balance`, `lem:side-degree`, `eq:family-count`, `eq:main-schedule`, and `app:optimization`.
- `eth_balanced_packing_2026-10-26.zip`; two-stage exponential-potential packing and integer capacity schedule.
- `eth_high_moment_2026-10-26.zip`; higher even moments, matching decomposition, quasipolynomial setup, and orbit-adaptive padding.
- `eth_packet_refinement_2026-10-26.zip`; matching-based derandomization and additive precision guard.
- `eth_recursive_refinement_2026-10-26.zip`; variance packing and residue-sensitive mode kernel.
- `3sat_round2_refinement_20261027.zip`; revised root/tail schedule and actual-domain research frontier.
- `eth_joint_cauchy_2026-10-26.zip`; full-orientation integral basis, cached contraction, and division-free matrix multiplication.

In the asymmetric packet's `code/balanced_packing.py`, the inspected implementation caches powers around lines 43–46 and future products around lines 64–69. Its candidate scoring around lines 79–85 performs the repeated row/cell scan replaced here. The balanced packet's `conditional_potential` and `derandomize_columns` instead reconstruct more of the expectation per candidate. Our stronger cached comparison avoids claiming novelty for optimizations already present in the asymmetric archive.

The exact cached schedule expressions and retuned two-stage comparison are recorded in `cache_schedule_new.md` and the accompanying schedule notes. Their evaluated coefficients are comparison bounds, not global optimization certificates.

The five closing archives are `sat_adaptive_contraction`, `sat_frontier_improvements`, `subexponential_sat_frontier`, `subexponential_sat_kernel_improvements`, and `subsat_contraction_tree`, each with suffix `_2026-10-26.zip`. Their exact hashes and closing commit are in `late_incoming/manifest.json`; substantive contributions and corrected attribution appear in Section 3.

The earlier report *Sharper quantitative bounds for subexponential SAT contraction*, its source and audit artifacts, remain a provenance record of the first iteration. Its $94.7482022595\ldots$ coefficient is explicitly superseded as the main generic comparator by the newer combined constructions. Neither that report nor the present one should be cited as a formal verification of the upstream claimed ETH refutation.

B Why the geometric allocation has its stated form

For a fixed tail alphabet, suppose the path masses are proportional to c^j and the available leading budget is B. The coefficient contribution is proportional to

$$\sum_{j\ge 0} \frac{c^j}{\sqrt{x_j}}, \qquad x_j > 0, \quad \sum_j x_j = B.$$

Lagrange multiplication, or the corresponding Holder inequality, gives $x_j \propto c^{2j/3}$. Substituting the normalized sequence yields

$$\min \sum_{j\ge 0} \frac{c^j}{\sqrt{x_j}} = \frac{1}{\sqrt{B}\,(1-c^{2/3})^{3/2}}.$$

This explains the two-stage comparator (4); it does not prove that a fixed tail alphabet is optimal when alphabets may vary. The explicit polynomial-space schedule chooses a rational ratio $3/4$, close to the natural tail allocation for $c = 2/3$, while keeping the depth bootstrap simple and strict.

An optimizer can suggest a sequence, but only the integer packing inequalities, precision proof, summable mass bound, and exact coefficient enclosure turn it into a theorem. This separation is especially important near excluded boundary budgets, where rounded feasibility checks can mistake an invalid equality for a permissible asymptotic schedule.